

Sentiment Analysis Based Stock Prediction

Nolan Smith
Emory University
Computer Science
Atlanta, GA USA
nolan.smith@emory.edu

Andrew Lee
Emory University
Computer Science
Atlanta, GA USA
andrew.lee3@emory.edu

Abstract—Stock Sentiment Analysis applies data mining and machine learning to Financial Markets, parsing text data to analyze overall sentiment. The research frames this as a classification task with two outputs: positive (bullish) and negative (bearish) sentiment. The model generates a polarity score between 0-1, 0 representing extremely negative sentiment, and 1 representing extremely positive sentiment. Text mining, NLP techniques, Data Preprocessing, Classification, and Clustering techniques are used to analyze all types of textual data from financial news to social media data.

Index Terms—Sentiment Analysis, Stock Prediction, Naive Bayes, Random Forest, Financial News, Machine Learning, Data Preprocessing, NLP, Classification, Clustering, Regression, Text Mining

I. INTRODUCTION

The prediction of stock prices has been a major challenge for all financial markets, with traditional methods relying heavily on technical analysis and historical price data. No matter what specific methods used, all investors wanted to find ways to be ahead of the buy-sell cycle. In recent years, the advent of social media and financial news has provided a new source of data. Not only has this data accumulated to pattern recognition extent, but it is greatly influential towards informing sentiment towards stock price prediction. This paper aims to investigate how sentiment analysis of financial statements can be used to predict stock price movements.

The primary motivation behind this research is the need for more efficient and accurate stock prediction tools. Should this research and model development show positive prediction rates, people may be able to earn more money with lower risk towards their savings: especially to lower experienced individuals. These individuals will trust the models because sentiment analysis is not a low level classifying system. Sentiment analysis is the extracted pattern of emotions and opinions from text data, which can then be used to show whether a stock is performing well or about to drop. This approach offers a more real-time and data-driven way of making stock predictions as opposed to traditional data analysis.

II. RELATED WORK

Sentiment analysis in working with stocks has been widely explored in many recent research studies. There have been

multiple types of articles based on varying types of datasets used. Chen et al. [1] conducted a bibliometric study on sentiment analysis for stock market research. This is a special variation on the sentiment analysis system as it focuses on basing sentiment analysis from previous articles. Most of the previous articles have used social media and other types of online content for their data. Xiao et al. [2] applied sentiment analysis to predict stock trends but specifically on Twitter posts mixed with other online posts. On the other hand, Joshi and Younes [3] focused on using news specific sentiment to predict stock price changes. These studies demonstrate the potential of sentiment analysis in stock prediction, although challenges such as data noise and class imbalance remain.

Other than the varying data, the methods used for sentiment analysis for predictions vary per experimenter as well. Palomo [4] conducted a study mainly focused on natural language processing models to predict the stock market. In addition, instead of creating classifiers from scratch, this paper was focused on using the premade tokenizer Financial BERT. Yet in a different study, Khonde et al. [5] focused on using vectorization to initially pull important phrases from news articles. The greatest takeaway is to take each of the most impactful aspects from each study to provide the most accurate prediction for the current dataset.

III. PROPOSED APPROACH

The proposed approach is to use sentiment analysis on two Kaggle data sets (1. Sentiment Analysis - Labeled Financial News Data 2. Financial Sentiment Analysis) to classify sentiment as positive (bullish), negative (bearish), which are then used as features for stock price prediction. The datasets were processed using pandas to merge and clean the data, ensuring that the information was suitable for sentiment analysis and machine learning model training. Other studies often contain the use of a neutral class. We decided against this approach as in finance, decisions often need to be binary i.e. buy or sell, risk-on or risk-off. Not only does this make it so we create decisive signals that are more aligned with real-world financial decision making, but also a lot of graphs and visualizations we created were unclear i.e. a 3x3 confusion matrix. The reason we used two datasets and these specific datasets are to strengthen the analysis. Using both allows the model to handle a range of text types, from short, concise sentences to longer news articles where sentiment can shift across paragraphs.

Short sentences help the model learn clear sentiment signals without the noise longer paragraphs bring. Longer texts expose it to more tone variations so the model can practice cutting through noise. Additionally, the biggest reason we used to datasets was that there were too many positive class entries in the first dataset so we wanted to balance it more and improve robustness, making the model better equipped to generalize across many types of financial text and prevent the model overfitting to one type of financial text style. The approach is then divided into the following stages.

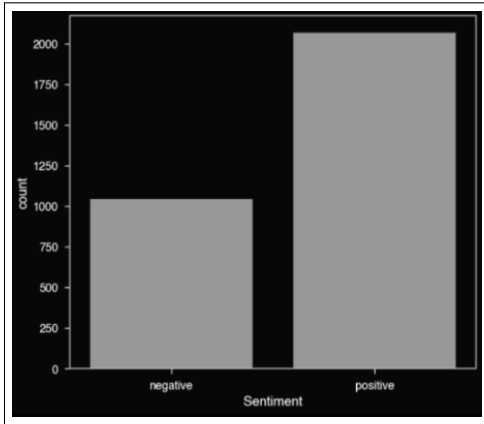


Fig. 1. This is a count plot showing class (Positive and Negative) imbalance.

A. Data Collection and Preprocessing

We used two datasets: one containing 400 labeled financial mini-articles and another with 2700 labeled financial sentences. The importance of the second dataset is the sheer amount of sentences and that they are all formatted similarly. This simplified dataset allows for the classification to be more robust. The datasets were merged to improve model generalization. Text preprocessing steps included lemmatization and stemming, which reduced noise and improved feature extraction. Other sources including the article "Predicting Stocks Changes Using News Sentiment Analysis" conclude that using just one method - lemmatization or stemming - is most effective, but with this suggestion we only increased accuracy by less than one percent. However, when used in tandem, we increased accuracy by three percent. This goes against previous research, but the results may be attributed to our specific financial data that had many different stems and suffixes of words. Applying both Porter's stemming algorithm and lemmatization drastically reduced noise and improved model performance. We applied these on the sentence column of our data frame. Lemmatization was used first to remove white space, punctuation, and reduce words to their root/lemma. An example of this process would be the transformation of the word investments turning into investment. Following lemmatization, Porter's stemming algorithm was applied to strip common suffixes like "es," "ing," "ed," and "s." This truncated words like transforming investment to invest. This dual process reduced the number of unique words by approximately 40 percent,

from 24,856 to 14,735, increasing generalization and making the dataset more focused.



Fig. 2. Word clouds before (top) and after (bottom) preprocessing with dual process of Porter's Stemming Algorithm and Lemmatization. In the bottom word cloud, fewer unique words are there, and many are shortened to their root or stem forms (i.e. companies, company → compani; inflate, inflation → inflat). The reduction in vocabulary size improves generalization by minimizing redundancy and emphasizing semantic patterns in our data.

B. Sentiment Classification

Before training the models, we experimented with different feature extraction techniques. We tried using Term Frequency (TF), Term Frequency Inverse Document Frequency (TF-IDF), and Count Vectorizers. Term Frequency counts how often a word appears in a document and gives higher weights to words that appear more frequently in documents and vice versa. TF-IDF adjusts weights by giving higher weights to rarer terms and is opposite of TF. In contrast, Count Vectorizer transforms the text into a sparse matrix of token counts and doesn't weight by document frequency. We found that Count Vectorizer significantly improved both model's performance and increased accuracy by approximately 5-10 percent compared to TF-IDF and TF. We believe Count Vectorizer outperformed TF and TF-IDF because of its ability to retain frequent financial terms without diluting importance through normalization or inverse weighting. With this in mind, we chose to use the Count Vectorizer for both our models. For sentiment classification, we used both the Naive Bayes and Random Forest classifier. The Naive Bayes model had a much higher accuracy and outperformed the Random Forest model. The performance gap can be attributed to the way Naive Bayes handles sparse matrices generated by our Count Vectorizer, especially when using unigrams and bigrams which allowed us to handle negation effectively. Before using bigrams as tokens, the sentence "Don't buy AMD" was given a positive rating because buy showed up a lot in our positive class entries. However, after using bigrams, the model understood that "don't buy" is a negative phrase and categorized the sentence as negative. With tens of thousands of features, Naive Bayes performs well by assuming feature independence and applying probabilistic

reasoning for estimation of likelihood for each class. It is better for identifying rare and impactful words and weighing them appropriately. To avoid issues like double underflow and to improve interpretability, we used the logarithmic form of the probability formula and applied Laplace smoothing to handle unseen words during inference. To implement Laplace smoothing, we added one to the numerator of each probability and the number of distinct words to the denominator of each probability. This was used to handle unseen words during inference.

In contrast, the Random Forest model consisted of 50 decision trees and used bootstrap sampling to improve generalization by creating varied training subsets. However, it struggled in the financial context as decision trees split on specific feature patterns and did not work as well for our sparse data. Also, sparse matrices reduce the effectiveness of information gain-based splitting, making it difficult for the model to capture meaningful decision thresholds and boundaries. Overall, Naive Bayes performed better and was the better choice for classifying the text into positive or negative classes.

C. Model Training and Evaluation

We experimented with several machine learning models, including Naive Bayes (NB) and Random Forest (RF). The models were trained using features extracted from the sentiment of the news articles. We evaluated the models using Monte-Carlo cross-validation, which involved 100 random splits of the data for training and testing.

IV. SYSTEM DESIGN

The system architecture consists of several key components:

- **Data Collection:** Data was collected using Python and pandas from Kaggle datasets.
- **Preprocessing:** The data was cleaned and preprocessed using NLP techniques such as tokenization, lemmatization, and stemming.
- **Sentiment Classification:** Sentiment scores were extracted using the Naive Bayes classifier.
- **Prediction Model:** Sentiment scores were used as input features for stock prediction models.

We implemented the system using Python, with libraries such as scikit-learn for machine learning, pandas for data manipulation, and nltk for text preprocessing. Additional libraries included the use of seaborn and matplotlib for data visualization and plotting graphs, WordCloud for visualizing word distributions, spacy for part of our lemmatization function, and NLTK's PorterStemmer and word_tokenize functions.

V. EXPERIMENT RESULTS

A. Sentiment Analysis Performance

The Naive Bayes classifier achieved an accuracy of 83 percent, outperforming other models such as Random Forest, which achieved 77 percent accuracy. Each model was evaluated using Monte-Carlo cross-validation, which provided a reliable estimate of its generalization ability.

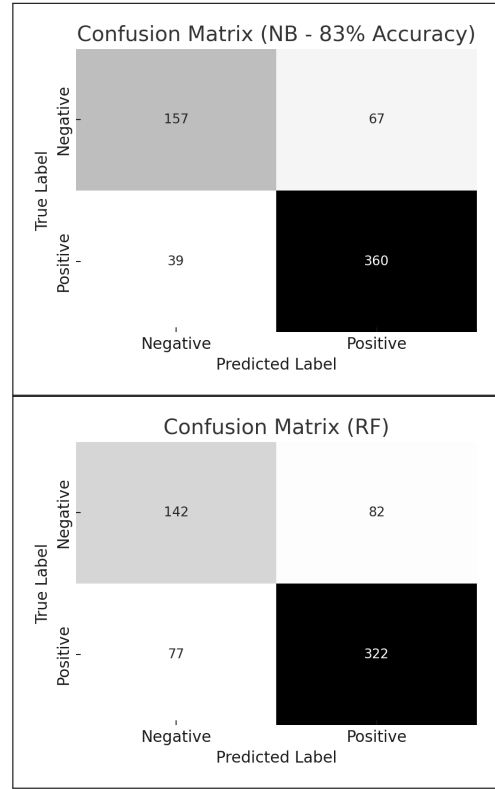


Fig. 3. Top shows confusion matrix of Naive Bayes model with 83 percent accuracy while bottom shows confusion matrix of Random Forest model with 77 percent accuracy

After multiple tests, we employed Monte-Carlo Cross-Validation to evaluate the performance of our models. We performed random sampling, where the dataset was split into 80 percent for training and 20 percent for testing through 100 iterations. This randomization ensures that the models are trained on diverse subsets of the data. By averaging the accuracy and other evaluation metrics over these 100 runs, we obtained a more reliable estimate of the model's performance. This methodology helped mitigate the variance from relying on a single training and testing split.

In terms of performance, the models exhibited higher specificity for the positive sentiment class compared to the negative sentiment class. This means the models were more effective at correctly identifying true negatives in positive sentiment instances but struggled more when classifying negative sentiment. The primary cause of these misclassifications is believed to be the class imbalance seen with the greater number of positive versus negative instances. This imbalance naturally led to the models being biased toward predicting positive sentiment, resulting in a higher specificity for the positive class.

Overall, the use of random sampling and Monte-Carlo Cross-Validation resulted in more generalized and reliable estimates of model performance, but it also revealed the need to address the class imbalance in future model iterations.

	precision	recall	f1-score
Negative	0.760	0.74	0.750
Positive	0.910	0.86	0.885
accuracy	0.830	0.83	0.830
macro avg	0.835	0.80	0.817
weighted avg	0.840	0.83	0.830

	precision	recall	f1-score
Negative	0.680	0.70	0.690
Positive	0.830	0.78	0.805
accuracy	0.770	0.77	0.770
macro avg	0.755	0.74	0.747
weighted avg	0.765	0.77	0.768

Fig. 4. NB clearly outperforms RF in every metric. Both NB and RF have higher precision and recall for the positive class which means that positive sentiment is more easily detected in financial text for our models.

B. Stock Prediction Performance

Using sentiment scores as input features, the models were able to predict stock trends with reasonable accuracy. The results indicated that sentiment analysis could help inform stock price predictions, but the overall prediction was not perfect. However, the Naive Bayes model was consistently more accurate compared to the other models tested.

VI. CONCLUSIONS

This paper demonstrates the feasibility of using sentiment analysis for stock market prediction. The Naive Bayes classifier performed well, with an accuracy of 83 percent, making it a valuable tool for stock prediction. Although However, challenges such as data noise and class imbalance need to be addressed to further improve prediction accuracy.

VII. CHALLENGES

Some of the main challenges encountered during this research include:

- Data quality issues especially with noisy text data
- Class imbalance in negative sentiment labels
- Lack of real-time data
- The dataset was still too small
- Difficulty in capturing nuanced sentiment from specific phrases
- Handling negation especially for phrases over 3 words in length
- Creating certain models especially the Naive Bayes model where we ran into double underflow, handling sparse matrices, and division by zero problems
- The models were underfitting and overfitting the data so we had to adjust parameters and hyperparameters

VIII. FUTURE APPROACHES

Future work could focus on improving the preprocessing steps, such as using more sophisticated NLP techniques, or

experimenting with deep learning models for sentiment analysis. Both "Sentiment Analysis for Stock Market Research: A Bibliometric Study" and Stock Trend Prediction Using Sentiment Analysis" articles in the references talk about the use of FinBERT, which is specifically fine-tuned on financial texts and understands word relationships in context. Unlike Naive Bayes or Random Forest, which relies on bag-of-words assumptions or manual feature engineering and contextual understanding, FinBERT understands sentiment shifts in financial jargon. To improve upon this study, incorporating FinBERT and comparing accuracy, generalization, and feature selection to Naive Bayes and Random Forest models would be a great step. Since FinBERT looks at word relationships and context, it would be interesting to see the improvement on our second dataset with longer texts and sentiment and tone shifts across paragraphs. In addition to the FinBERT model, we can evaluate the performance of large language models (LLMs) such as advanced GPT models such as GPT 4.5 or traditional NLP models like Stanford CoreNLP. Like FinBERT, these models can capture subtle sentiment shifts and contextual cues that the models we used can't due to them being based on feature representations. It would be best if the model continued to use the most accurate Naive-Bayes classifier because of its high accuracy with the current data. On the note of data, using more diverse data sources, such as social media posts, news articles, or academic articles could provide a more comprehensive view of market sentiment.

ACKNOWLEDGMENT

The authors would like to thank Professor Shu for the opportunity to learn and grow a powerful working sentiment analysis model.

REFERENCES

- [1] X. Chen, H. Xie, Z. Li, H. Zhang, X. Tao, and F. L. Wang, "Sentiment Analysis for Stock Market Research: A Bibliometric Study," *Natural Language Processing Journal*, vol. 10, Mar. 2025. [Online]. Available: <https://doi.org/10.1016/j.nlp.2025.100125>.
- [2] X. Qiao and B. Ihnaini, "Stock Trend Prediction Using Sentiment Analysis," *PeerJ Comput. Sci.*, vol. 9, e1293, Mar. 2023. [Online]. Available: <https://doi.org/10.7717/peerj-cs.1293>.
- [3] A. Joshi and R. Younes, "Predicting Stocks Changes Using News Sentiment Analysis," SSRN, 2023. Available: <https://ssrn.com/abstract=4605587>.
- [4] C. Palomo, "Tweet Sentiment Analysis to Predict Stock Market," [Online]. Available: <https://web.stanford.edu/class/archive/cs/cs224n/cs224n.1234/final-reports/final-report-170049613.pdf>
- [5] Khonde, S.R., Virmodkar, S.S., Nemade, S.B., Dudhedia, M.A., Kanawade, B., Gawande, S.H. (2024). Sentiment analysis and stock data prediction using financial news headlines approach. *Revue d'Intelligence Artificielle*, Vol. 38, No. 3, pp. 999-1008. <https://doi.org/10.18280/ria.380325>